

Object-Oriented Analysis and UML-Based Design of an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS)

Question / Problem Statement

Course: CSA11 – Object Oriented Analysis and Design | **Course Outcomes:** CO1, CO2, CO3

CO1: Analyse Object-Oriented Systems Development concepts, methodologies, life cycle models, Model-Driven Development (MDD), and Agile practices for developing a real-world software solution.

CO2: Apply UML modelling techniques to represent system structure, interactions, workflows, behaviour, components, and deployment architecture.

CO3: Analyse objects, classes, attributes, methods, and relationships using Use Case Modelling, Object Analysis, Classification, and relationship identification techniques.

SDG Mapping: SDG 3 – Good Health and Well-being; SDG 9 – Industry, Innovation and Infrastructure; SDG 11 – Sustainable Cities and Communities; SDG 13 – Climate Action.

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) that supports emergency teams in locating affected people, assessing disaster zones, coordinating rescue operations, and monitoring emergency situations. The system shall allow authorized emergency personnel to register disaster incidents, define affected zones, assign drones, monitor drone status, and manage rescue missions. Autonomous drones equipped with cameras and sensors shall collect information about disaster locations and transmit images, videos, environmental readings, and location information to the central command system. The system shall maintain information about drones, operators, rescue missions, disaster zones, victims, emergency teams, sensor data, alerts, and resources. The system shall support route planning, mission assignment, real-time drone monitoring, emergency notifications, and mission completion reporting. Working from this scenario, students shall identify the major actors and use cases and develop Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams. Students shall identify object responsibilities and evaluate suitable design principles and patterns for achieving low coupling, high cohesion, flexibility, and maintainability.

Constraints

The design shall clearly identify the system boundary, emergency actors, autonomous components, objects, classes, attributes, methods, and relationships. All UML diagrams shall maintain consistency with the identified use cases and system behaviour. The model shall explicitly consider real-time communication, autonomous operations, failure handling, security, reliability, scalability, and responsibility assignment. Design axioms and suitable design patterns shall be applied wherever appropriate. The final submission shall include assumptions, constraints, traceability between use cases and classes, and justification for major design decisions.

Answer

1. Introduction

The Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) is a real-time, safety-critical software system that coordinates human emergency responders and autonomous aerial drones during disaster situations such as floods, earthquakes, fires, and building collapses. The system authenticates emergency personnel, registers disaster incidents, defines affected zones, plans safe drone routes, launches and monitors autonomous drones, collects sensor and camera data, detects victims, generates alerts, coordinates rescue teams, manages resources, and produces mission reports. Because ADDRRS combines human decision-making with autonomous, sensor-driven robotic behaviour operating over unreliable wireless links, an Object-Oriented Analysis and Design (OOAD) approach is well suited: it allows the domain to be modelled as a set of collaborating objects (drones, missions, zones, sensors, alerts) whose responsibilities, states, and interactions can be captured precisely using UML.

This document analyses the ADDRRS domain end-to-end — covering actors and use cases, object/class analysis, all eight required UML diagrams, responsibility assignment, design principles and patterns, assumptions, constraints, traceability, non-functional qualities, the development methodology, and a final reflection — in line with CO1, CO2, and CO3.

2. Requirements Analysis

2.1 Functional Requirements

- The system shall authenticate and authorize emergency personnel, drone operators, and administrators before granting access.
- The system shall allow authorized users to register a disaster incident and define the affected geographic zone.
- The system shall allow assignment of an available autonomous drone to a rescue mission and shall plan a safe route to the disaster zone.
- The system shall launch drones, monitor their real-time telemetry (position, battery, status), and allow rescue coordinators to re-plan routes if conditions change.
- Autonomous drones shall collect camera, GPS, and environmental sensor data and transmit it to the command centre for analysis.
- The system shall analyse incoming sensor data to detect victims and shall generate emergency alerts automatically when a victim is located.
- The system shall notify and coordinate rescue teams, allow them to confirm a rescue, and track mission progress until completion.
- The system shall handle drone failures (low battery, communication loss) through automated recovery actions such as emergency landing or auto-return.
- The system shall manage shared resources (drones, rescue teams, equipment) and avoid conflicting assignments.
- The system shall generate a mission completion report summarizing route, sensor findings, victims located, and rescue outcome.

2.2 Non-Functional Requirements

- **Reliability:** the system must tolerate drone communication loss, sensor failure, and low-battery conditions without losing mission data.
- **Real-time responsiveness:** telemetry, alerts, and victim-detection notifications must reach the command dashboard with minimal latency.

- **Security:** all channels (dashboard, mobile app, drone links) must be authenticated and encrypted (TLS/HTTPS) to prevent unauthorised mission control.
- **Scalability:** the architecture must support many concurrent missions, drones, and rescue teams across multiple simultaneous disaster zones.
- **Usability:** the command dashboard and mobile rescue-team interface must present mission status clearly under high-stress conditions.
- **Maintainability:** the system shall be modular (packages/components) so that new sensor types, drone models, or notification channels can be added with minimal impact on existing code.

3. Actors

Actors represent roles external to the ADDRRS system boundary that initiate or participate in system behaviour. The Use Case Diagram (Figure 1) identifies the following primary and secondary actors, consistent with the Sequence Diagram (Figure 3) and Deployment Diagram (Figure 8):

Actor	Type	Role in the System
Emergency Commander	Primary (human)	Authenticates, registers incidents, creates and assigns rescue missions, monitors overall operations, and reviews mission reports.
Drone Operator	Primary (human)	Registers/prepares drones, plans routes, launches missions, and monitors drone telemetry and health.
Rescue Team Member	Primary (human)	Receives alerts, travels to located victims, confirms rescue, and completes the mission.
System Administrator	Primary (human)	Manages user accounts, security policy, drone registration, and system configuration.
Autonomous Drone	Secondary (system)	Executes flight plans, collects sensor/camera data, transmits telemetry, and reacts to on-board failure conditions.
External Emergency / Notification Service	Secondary (system)	Receives outbound alerts/notifications (SMS, push, email) triggered by the Alert Manager.

4. Use Case Modelling

The Use Case Diagram captures the functional scope of ADDRRS as seen by its actors. Core use cases include *Authenticate User*, *Register Incident*, *Define Disaster Zone*, *Create Rescue Mission*, *Plan Route*, *Launch Mission*, *Monitor Drone*, *Collect Sensor Data*, *Locate Victim*, *Generate Alert*, *Coordinate Rescue Team*, *Manage Resources*, *Handle Drone Failure*, *Complete Mission*, and *Generate Mission Report*. **<<include>>** relationships model mandatory sub-behaviour — for example, *Create Rescue Mission* always includes *Plan Route*, and *Launch Mission* always includes *Register Incident/Plan Route* pre-conditions, while *Locate Victim* always includes *Collect Sensor Data*. **<<extend>>** relationships model optional/conditional behaviour — *Handle Drone Failure* extends *Manage Resources/Coordinate Rescue Team* only when a fault condition arises during a mission.

UML Use Case Diagram — Autonomous Drone-Based Disaster Response and Rescue System

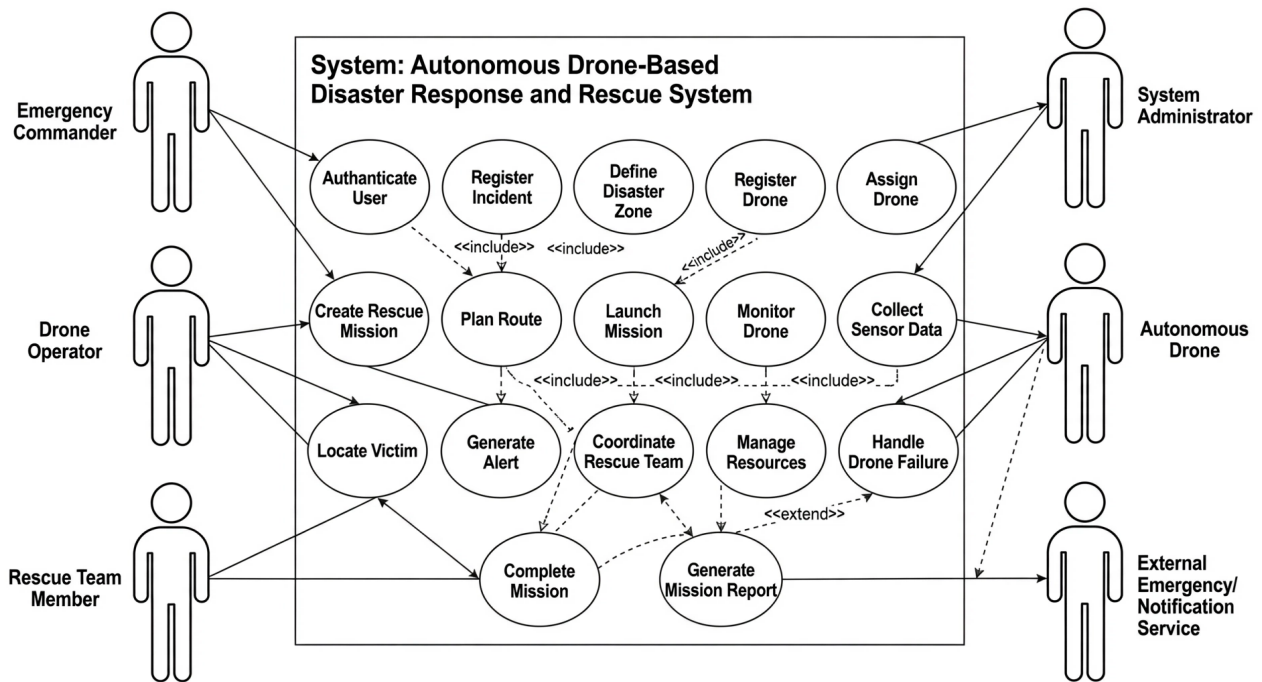


Figure 1: Use Case Diagram

5. Object and Class Analysis

Applying noun-phrase and responsibility-driven analysis to the problem statement, the following key domain classes were identified, each with representative attributes and methods derived directly from the use cases and the Sequence/State/Activity diagrams.

Class	Key Attributes	Key Methods
User (abstract)	userId, name, role, credentials	login(), logout(), authenticate()
EmergencyCommander	commanderId, rank, jurisdiction	createMission(), assignDrone(), reviewReport()
DroneOperator	operatorId, certification	planRoute(), launchDrone(), monitorDrone()
RescueTeamMember	teamId, location, status	receiveAlert(), confirmRescue(), completeMission()
SystemAdministrator	adminId, accessLevel	manageUsers(), registerDrone(), configureSystem()
DisasterIncident	incidentId, type, timestamp, severity	registerIncident(), updateStatus()
DisasterZone	zoneId, coordinates, boundary, riskLevel	defineZone(), getBoundary()
RescueMission	missionId, status, startTime, endTime, priority	createMission(), validateMission(), assignDrone(), completeMission()
MissionController	controllerId, activeMissions	validateMission(), planSafeRoute(), coordinate()
RoutePlanner	routeId, waypoints, distance	planRoute(), replanRoute(), optimisePath()
Drone (Autonomous Drone)	droneId, model, batteryLevel, gpsPosition, status	preFlightCheck(), navigateToZone(), collectSensorData(), returnToBase(), handleFailure()
Sensor	sensorId, type, reading, unit	capture(), transmitReading()
SensorData	dataId, timestamp, imageUrl, gpsCoord, reading	collect(), analyse(), store()
Victim	victimId, location, condition, detectedTime	recordDetection(), updateStatus()
AlertManager	alertId, severity, message, timestamp	generateAlert(), notifyTeam(), escalate()
CommandCenter	centerId, activeDashboards	updateDashboard(), aggregateTelemetry()
Resource	resourceId, type, availability	allocate(), release()
MissionReport	reportId, summary, victimsFound, generatedAt	generateReport(), exportReport()

5.1 Classification and Object Relationships

Association: EmergencyCommander is associated with RescueMission (creates/oversees); DroneOperator is associated with Drone (operates). **Aggregation (has-a, independent lifecycle):** RescueMission aggregates Resources and a RescueTeamMember roster — these can exist and be reassigned independently of any single mission. **Composition (part-of, dependent lifecycle):** RescueMission is composed of a Route and a set of SensorData records — route and sensor readings belong exclusively to their mission and are destroyed with it. **Dependency:** MissionController depends on RoutePlanner and AlertManager to fulfil a request without owning them. **Generalisation/Inheritance:** EmergencyCommander, DroneOperator, RescueTeamMember, and SystemAdministrator all generalise from an abstract User superclass, sharing authentication behaviour while specialising role-specific operations. **Realisation:** Drone realises a Navigable/Sensable interface exposing navigateToZone() and collectSensorData() to the MissionController without exposing internal flight-control logic.

6. UML Diagrams

The eight UML diagrams below model ADDRRS from complementary perspectives — structural, behavioural, and architectural — in line with CO2. Each diagram is consistent with the actors, use cases, and classes identified above.

6.1 Use Case Diagram

Presented above as Figure 1, this diagram defines the system boundary and the functional interactions between actors and the ADDRRS system.

6.2 Sequence Diagram — Create and Execute Rescue Mission

The Sequence Diagram traces the complete lifecycle of a rescue mission as time-ordered messages between objects: the Emergency Commander authenticates and creates a mission; the MissionController validates it and invokes the RoutePlanner (a self-call plans the safe route); the Drone performs a pre-flight self-check, is launched, and navigates to the zone; the Drone and Sensor collect and transmit telemetry to the Command Center, which updates the monitoring dashboard; upon victim detection the AlertManager generates an alert and notifies the Rescue Team; the team confirms the rescue; the Drone returns to base; and finally the mission is completed and a MissionReport is generated and returned to the Commander. This diagram directly validates the <<include>> relationships shown in the Use Case Diagram.

UML Sequence Diagram - Create and Execute Rescue Mission (ADDRRS)

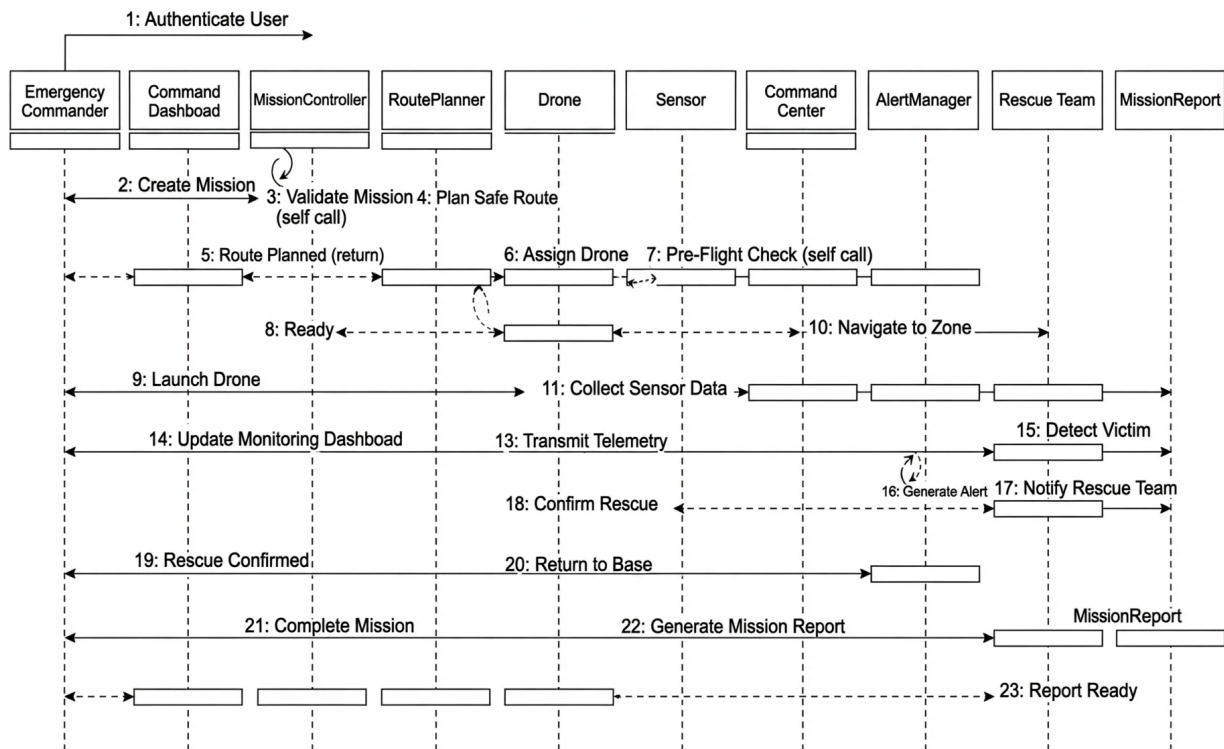


Figure 2: Sequence Diagram — Create and Execute Rescue Mission

6.3 Activity Diagram — Disaster Rescue Workflow

The Activity Diagram models the end-to-end workflow logic, including the two key decision points in the system: *Victim Detected?*, which loops the drone back into Collect/Analyse Data when no victim is found and otherwise proceeds to Generate Alert; and *Rescue Completed?*, which loops Monitor Rescue while the rescue is in progress and otherwise proceeds to Return Drone, Store Mission Data, and Generate Mission Report. This complements the Sequence Diagram by showing the branching/looping control flow rather than individual object messages.

UML Activity Diagram - Disaster Rescue Workflow (ADDRRS)

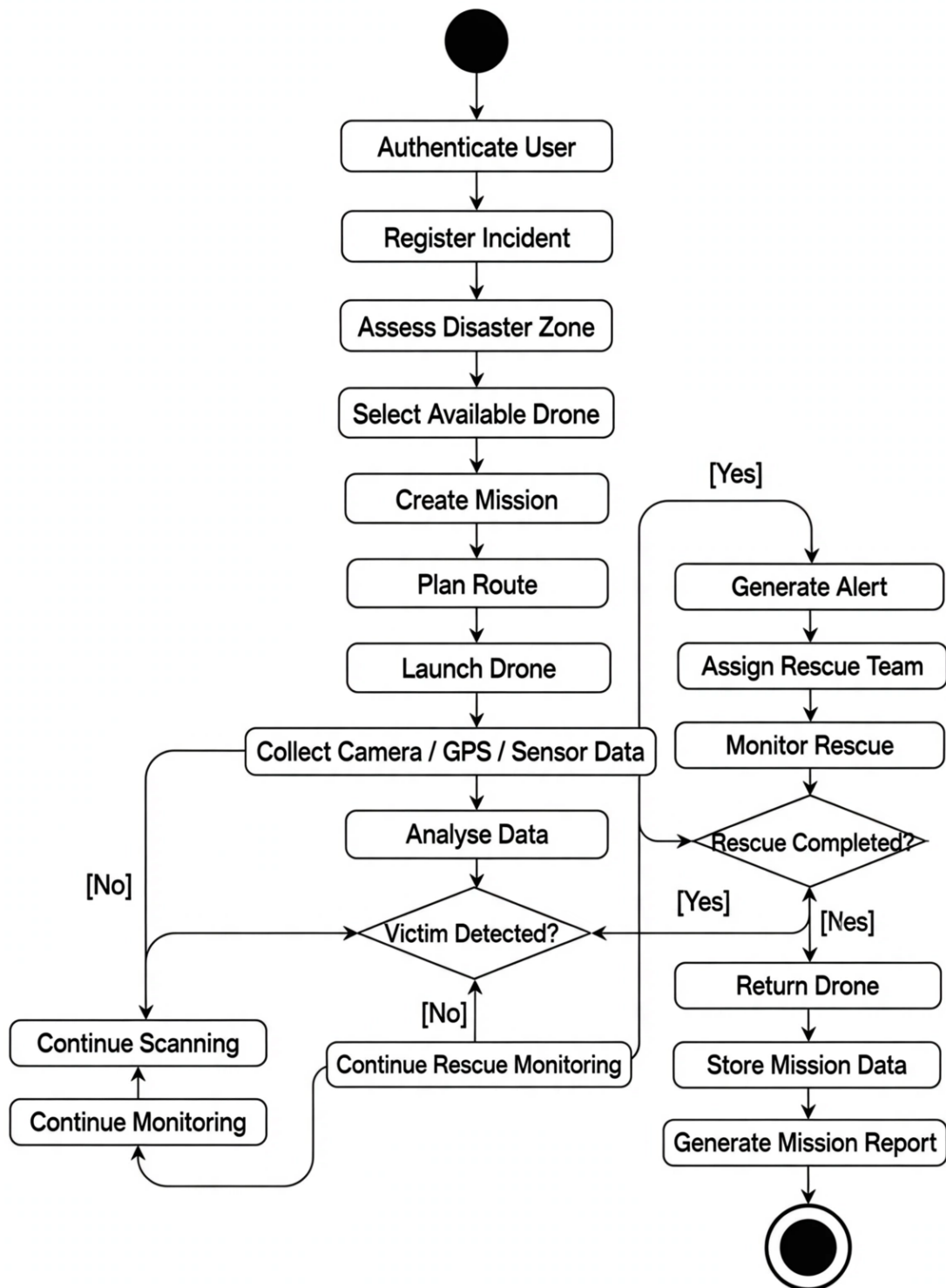


Figure 3: Activity Diagram — Disaster Rescue Workflow

6.4 State Transition Diagram — Autonomous Drone

The State Diagram models the lifecycle of the most behaviourally complex object in the system, the Drone. From *Idle*, a drone becomes *Assigned*, undergoes *PreFlightCheck*, *Launching*, and *EnRoute*, reaching *VictimLocated* and *RescueSupport* before *Returning* and finally *MissionComplete*. Two internal-transition guards are modelled globally: from any state, *battery low* transitions to *LowBattery* and, if it worsens, to *EmergencyLanding* (a final state), while *critical failure* also routes to emergency landing; similarly, a lost communication link transitions *Launching* to *CommunicationLost*, which either recovers/reconnects back to *Launching* or times out into an automatic *EmergencyLanding*/auto-return to *MissionComplete*. This diagram is the behavioural basis for the *Handle Drone Failure* use case.

UML State Diagram - Autonomous Drone (ADDRRS)

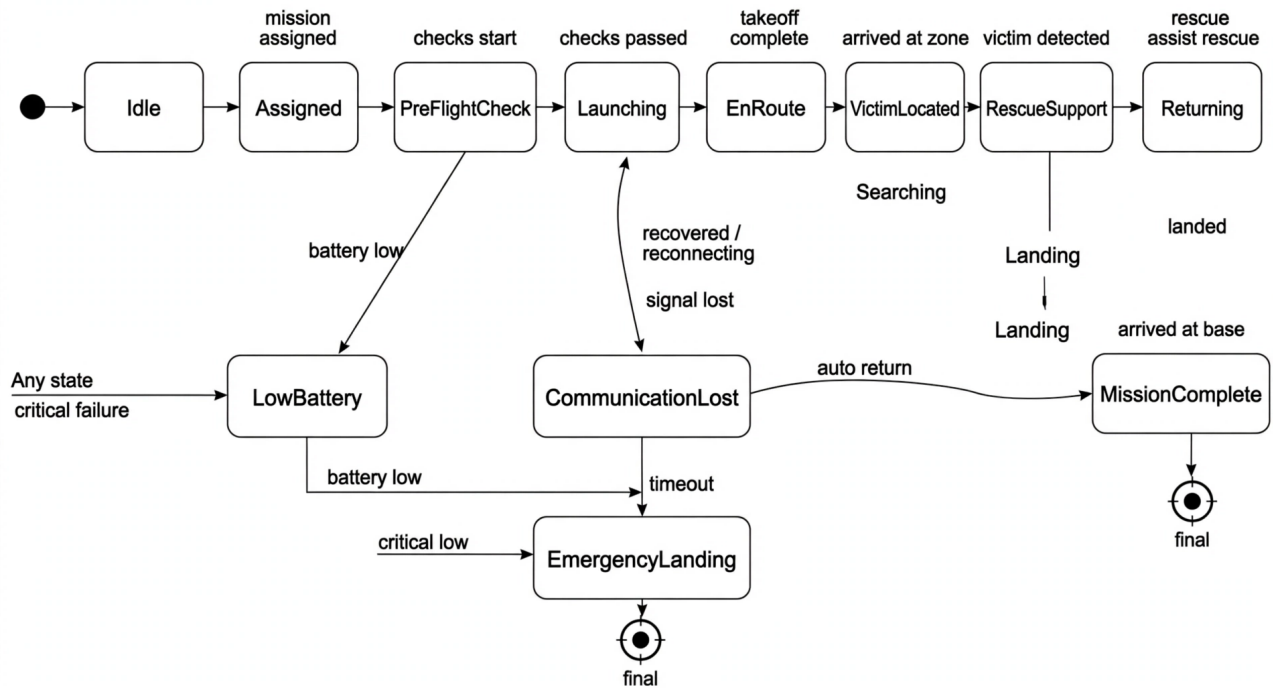


Figure 4: State Transition Diagram — Autonomous Drone

6.5 Package Diagram

The Package Diagram groups related classes into cohesive, loosely-coupled subsystems: *User Management* and *Security* (protected by/protecting other packages), *Incident Management*, *Mission Management* (the central package, depending on Sensor Data, Resource Management, Drone Management, and Security, and depended on by Rescue Management), *Drone Management*, *Sensor Data*, *Reporting*, *Resource Management*, *Rescue Management*, and *Alert and Notification*. This grouping supports the low-coupling/high-cohesion goal required by the assignment and maps directly onto the services shown in the Component Diagram.

UML Package Diagram - ADDRRS

Autonomous Drone-Based Disaster Response and Rescue System

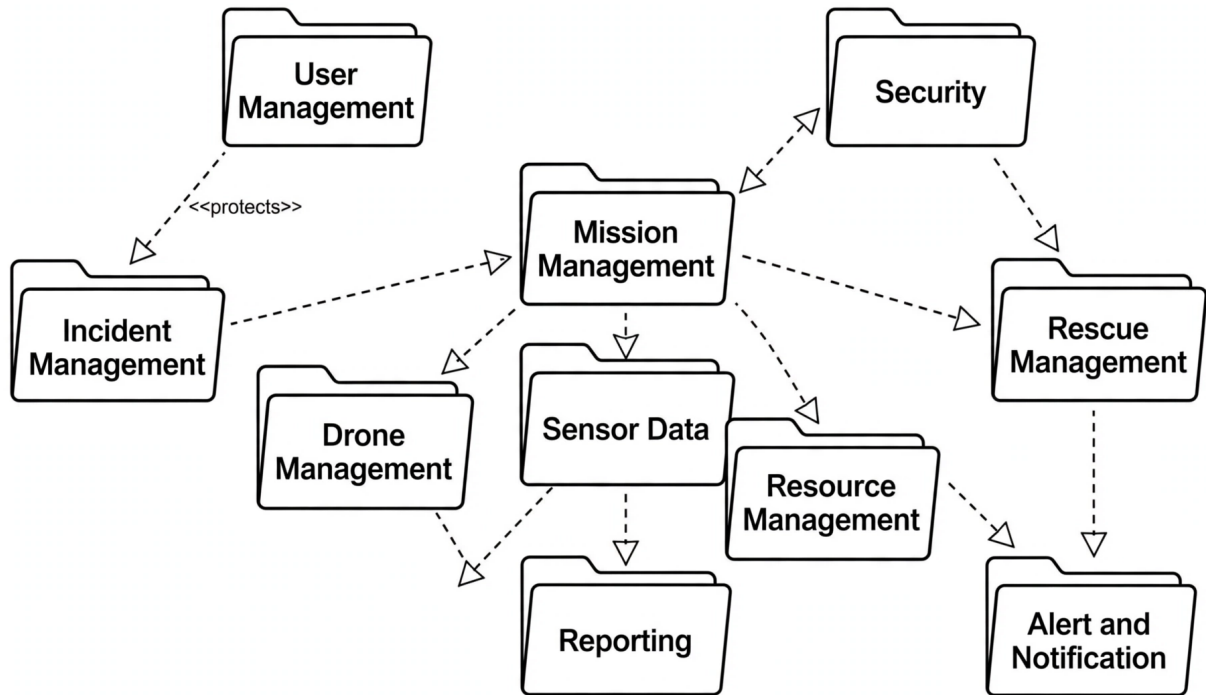


Figure 5: Package Diagram

6.6 Component Diagram

The Component Diagram refines the package structure into deployable software components and their <<use>> dependencies: front-end components (*Command Dashboard*, *Rescue Mobile Interface*) use back-end services (*Authentication Service*, *Mission Service*) via the *API Gateway/Incident Service*; the *Drone Control Service* and *Route Planning Service* use the *Drone Communication Gateway* and *Telemetry Ingestion Service*; telemetry flows through a *Message Broker* to the *Alert Service* and *Notification Gateway*; and the *Resource Service* and *Reporting Service* persist to a shared *Database* and *Object Storage*. This diagram shows how the packages above are realised as concrete, independently deployable services.

UML Component Diagram - ADDRRS

Autonomous Drone-Based Disaster Response and Rescue System

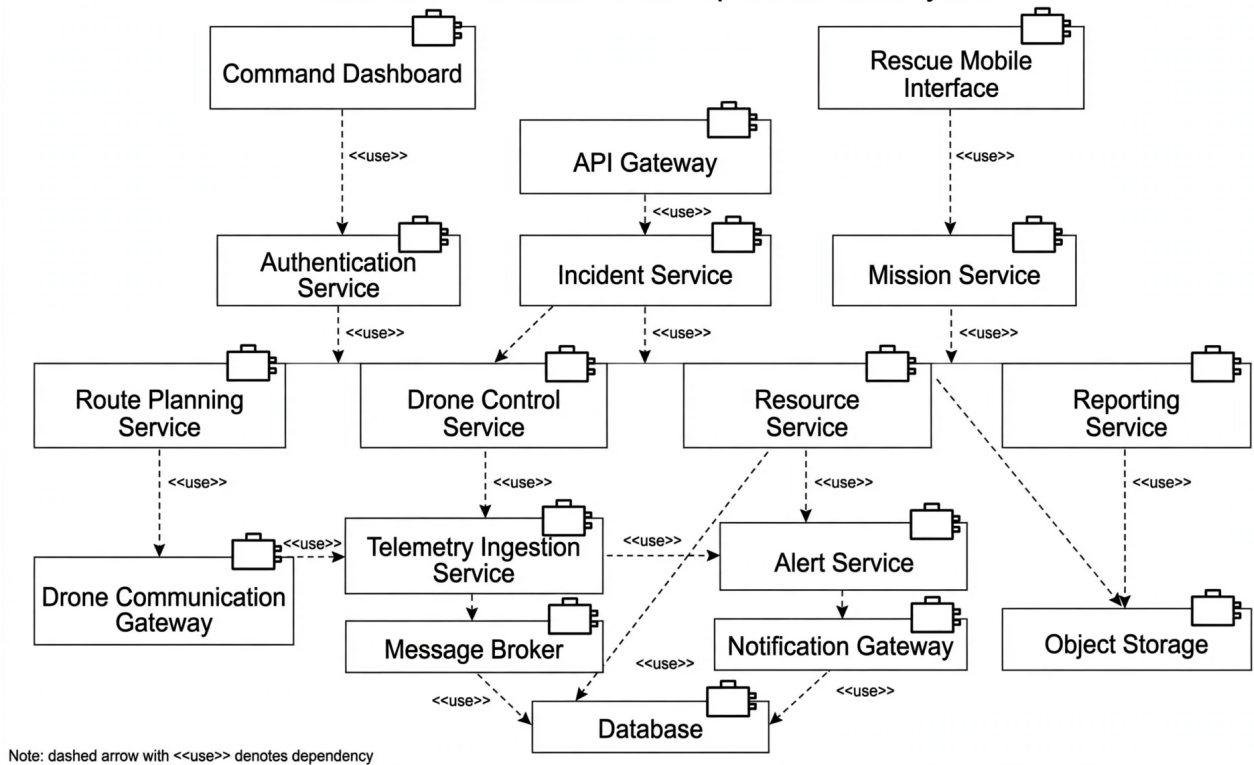
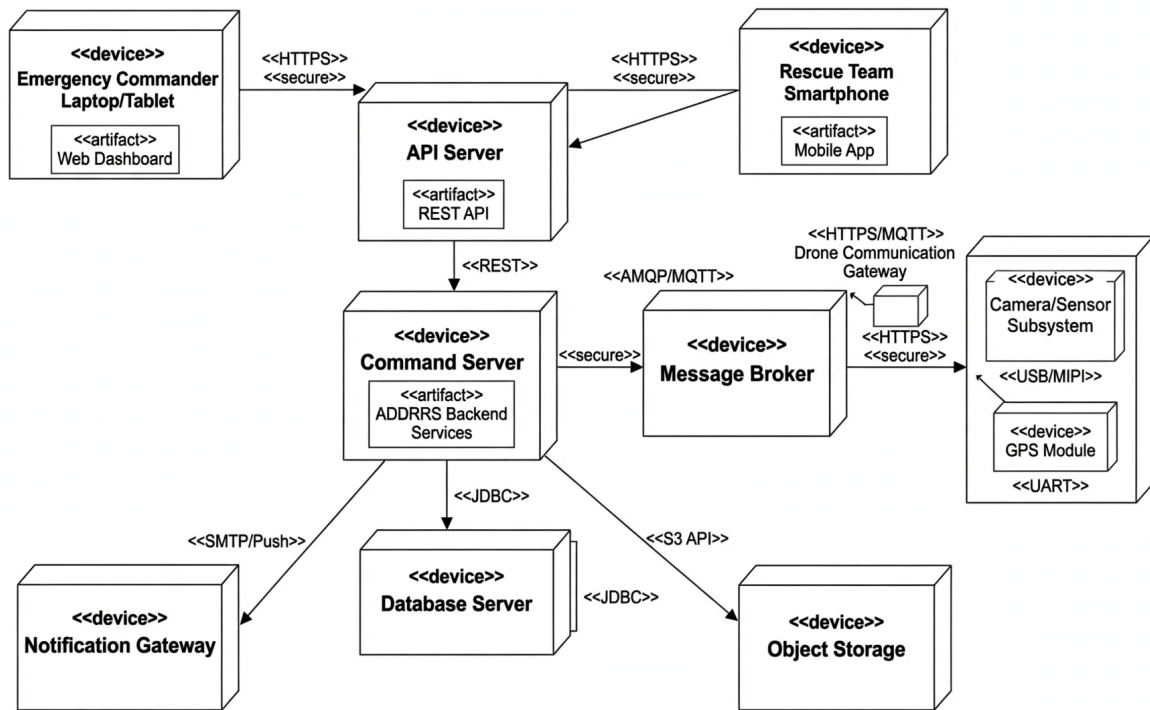


Figure 6: Component Diagram

6.7 Deployment Diagram

The Deployment Diagram shows the physical/network topology: the Emergency Commander's laptop/tablet (running the Web Dashboard artifact) and the Rescue Team smartphone (running the Mobile App artifact) both connect over secure HTTPS to the API Server (hosting the REST API artifact), which forwards requests over REST to the Command Server (hosting the ADDRRS Backend Services). The Command Server persists data to the Database Server via JDBC and to Object Storage via the S3 API, sends notifications via SMTP/Push to the Notification Gateway, and exchanges telemetry with a Message Broker over AMQP/MQTT. The Message Broker communicates with the Drone Communication Gateway over secure HTTPS/MQTT, which in turn bridges to the on-drone Camera/Sensor Subsystem (with an on-board GPS Module over UART) via USB/MIPI. All links marked <<secure>> are TLS-protected, directly addressing the assignment's security constraint.

UML Deployment Diagram - ADDRRS



Notes: All communications marked <<secure>> use TLS. Stereotypes denote protocol.

Figure 7: Deployment Diagram

7. Object Responsibilities

Responsibility-driven design (CRC-style analysis) was used to assign each class a focused set of responsibilities and collaborators, keeping responsibilities aligned with the GRASP principles discussed in Section 8.

Class	Responsibilities	Key Collaborators
MissionController	Validate mission requests; coordinate route planning and drone assignment; orchestrate the mission lifecycle.	RoutePlanner, Drone, RescueMission
RoutePlanner	Compute a safe route to the disaster zone; re-plan on obstacle or hazard detection.	MissionController, Drone
Drone	Execute the flight plan; run pre-flight checks; collect and transmit sensor data; detect and react to failure conditions.	Sensor, MissionController, CommandCenter
AlertManager	Decide alert severity; generate and route alerts to the correct rescue team.	SensorData, RescueTeamMember, Notification Gateway
CommandCenter	Aggregate telemetry from multiple drones; refresh dashboards in real time.	Drone, MissionController
MissionReport	Compile the mission summary, sensor findings, and outcome for archival and audit.	RescueMission, SensorData

8. Design Principles and Patterns

8.1 Design Axioms and Corollaries

Axiom 1 (Independence Axiom): maintain the independence of functional requirements — applied by separating Mission Management, Drone Management, Sensor Data, and Alert/Notification into distinct packages so a change in alert delivery does not ripple into route planning. **Axiom 2 (Information Axiom):** minimise information content/complexity — applied by keeping each class's attribute set minimal and delegating cross-cutting concerns (e.g., authentication) to a single shared Security package rather than duplicating it in every actor class.

8.2 GRASP Principles

- **Information Expert:** Drone owns telemetry and is therefore assigned `collectSensorData()/handleFailure()`; RouteFinding logic is assigned to RoutePlanner, which has the map/waypoint data.
- **Creator:** MissionController creates RescueMission objects because it aggregates and initialises the mission's constituent parts (route, drone assignment).
- **Controller:** MissionController acts as the coordinating controller for use cases such as Create Mission and Handle Drone Failure, decoupling the UI (Command Dashboard) from domain logic.
- **Low Coupling:** the Drone communicates with MissionController and CommandCenter through well-defined interfaces rather than directly manipulating RescueMission internals.
- **High Cohesion:** AlertManager is solely responsible for alert generation/routing; it does not also manage resources or routes.
- **Polymorphism:** the abstract User class allows role-specific `authorize()` behaviour to vary by subclass (Commander, Operator, Team Member, Administrator) without conditional branching in client code.
- **Protected Variations:** the Drone Communication Gateway shields the rest of the system from changes in drone hardware/protocol (MQTT vs HTTPS), and the interface-based Sensor abstraction allows new sensor types to be added transparently.

8.3 Design Patterns Applied

- **Observer Pattern:** the CommandCenter/Command Dashboard subscribes to Drone telemetry updates and AlertManager notifications, enabling real-time monitoring without polling.
- **State Pattern:** the Drone's behaviour (PreFlightCheck, EnRoute, LowBattery, CommunicationLost, EmergencyLanding, etc., as modelled in Figure 4) is implemented using the State pattern so each state encapsulates its own allowed transitions and actions.
- **Strategy Pattern:** RoutePlanner can select among alternative routing/obstacle-avoidance algorithms (e.g., shortest-path vs. hazard-avoiding) interchangeably at runtime.
- **Mediator Pattern:** MissionController mediates interactions between RoutePlanner, Drone, AlertManager, and RescueTeamMember, preventing direct many-to-many coupling between these classes.
- **Facade Pattern:** the API Gateway (Figure 6) provides a simplified, unified entry point to Incident, Mission, and Drone services for client applications.
- **Publish-Subscribe (Message Broker) Pattern:** telemetry and alert events are published to a Message Broker (Figures 6 and 8) and consumed asynchronously by the Alert Service and Notification Gateway, decoupling producers from consumers and improving scalability.
- **Singleton (qualified use):** a single MissionController/CommandCenter instance per deployment coordinates active missions, avoiding conflicting state.

9. Assumptions and Constraints

9.1 Assumptions

- Each drone is assigned to at most one active rescue mission at a time.
- Drones have sufficient onboard battery and computing capacity to perform pre-flight checks and limited autonomous decision-making (e.g., emergency landing) without a live network connection.
- Emergency personnel, drone operators, and rescue team members have valid, pre-provisioned credentials issued by a System Administrator.
- Network connectivity between the drone and the Drone Communication Gateway may be intermittent; the design must tolerate brief signal loss.
- Sensor data (image/video/GPS/environmental readings) is analysed using an onboard or edge victim-detection capability that reports a boolean/confidence result to the Mission Controller.
- GPS coordinates are the standard spatial reference used across DisasterZone, Route, and Victim location data.

9.2 Constraints

- The system boundary is limited to mission coordination, drone control orchestration, sensor-data handling, alerting, and reporting; it does not include low-level flight-control firmware, which is treated as part of the Drone component.
- All UML diagrams must remain consistent with the identified use cases (traceability enforced in Section 10).
- Communication with drones must be treated as unreliable; every drone interaction must define a failure/timeout path (see State Diagram, Figure 4).
- All external communication channels must be secured using TLS/HTTPS as shown in the Deployment Diagram.
- The design must support horizontal scaling of drone fleets and concurrent missions without architectural rework.

10. Traceability Between Use Cases and Classes

The table below demonstrates traceability from major use cases (Figure 1) to the classes and diagrams that realise them, satisfying the constraint that all UML views remain consistent with the identified use cases.

Use Case	Primary Classes Involved	Diagram(s) Illustrating It
Authenticate User	User, EmergencyCommander, SystemAdministrator	Use Case (Fig.1), Sequence (Fig.2), Deployment (Fig.7)
Create Rescue Mission / Plan Route	MissionController, RescueMission, RoutePlanner	Use Case, Sequence, Activity (Fig.3)
Launch Mission / Monitor Drone	Drone, CommandCenter	Sequence (Fig.2), State (Fig.4)
Collect Sensor Data / Locate Victim	Sensor, SensorData, Victim	Sequence, Activity, Component (Fig.6)
Generate Alert / Coordinate Rescue Team	AlertManager, RescueTeamMember	Sequence, Activity, Component
Handle Drone Failure	Drone, MissionController	State (Fig.4), Activity
Manage Resources	Resource, MissionController	Package (Fig.5), Component
Complete Mission / Generate Mission Report	RescueMission, MissionReport	Sequence, Activity, Component

11. Security, Reliability, and Scalability

11.1 Security

Security is addressed structurally through a dedicated Security package (Figure 5) that protects User Management and is used by Mission Management, and through an Authentication Service component (Figure 6) that gates all dashboard and mobile-app requests. At the network level (Figure 8), every external link — Commander laptop/tablet to API Server, Rescue Team smartphone to API Server, Command Server to Message Broker, and Message Broker to the Drone Communication Gateway — is marked <<secure>> and runs over HTTPS/TLS, preventing interception or spoofing of mission-critical commands and telemetry. Role-based authorization (via the User generalisation hierarchy) ensures a Rescue Team Member cannot, for example, issue drone launch commands reserved for a Drone Operator or Emergency Commander.

11.2 Reliability

Reliability is primarily modelled in the State Diagram (Figure 4): a drone that loses communication moves to `CommunicationLost` and either reconnects or, on timeout, proceeds to an automated `EmergencyLanding/auto-return` path, so a single link failure cannot strand a mission with no recovery action. Low battery is handled similarly as a state reachable from any state, guaranteeing the drone always has a safe, well-defined fallback. The Activity Diagram's `Rescue Completed?` and `Victim Detected?` decision loops ensure the workflow keeps monitoring rather than terminating prematurely on inconclusive sensor data.

11.3 Scalability

The Component Diagram's use of a Message Broker for telemetry and alert propagation (publish-subscribe) decouples the rate at which drones produce telemetry from the rate at which the dashboard, Alert Service, and Reporting Service consume it, allowing many concurrent drones and missions to be supported by scaling broker/consumer instances independently. The package structure (Figure 5) further allows Drone Management, Mission Management, and Rescue Management to be scaled and deployed as independent services (Figure 6), consistent with a microservice-oriented evolution of the architecture.

12. Development Methodology (CO1)

12.1 Object-Oriented Systems Development (OOSD) Life Cycle

ADDRRS was analysed using the classical OOSD life cycle: **(1) Object-Oriented Analysis (OOA)** — identifying actors, use cases, domain objects, and their responsibilities (Sections 3–5, 7); **(2) Object-Oriented Design (OOD)** — refining the analysis model into classes with attributes, methods, relationships, states, and applying design principles/patterns (Section 8) and the structural diagrams (Package, Component, Deployment); **(3) Object-Oriented Programming (OOP)** — implementing the designed classes/components (backend services shown in Figure 6) in an OO language; and **(4) Testing and Iteration** — verifying each use case against its Sequence/Activity/State behaviour and iterating the model as new drone or sensor capabilities are discovered. Because ADDRRS behaviour is safety-critical and only partially known up front (exact drone failure modes, sensor accuracy), the life cycle is inherently iterative and incremental rather than a single-pass waterfall.

12.2 Model-Driven Development (MDD)

The eight UML diagrams produced here form the Platform-Independent Model (PIM) of ADDRRS: the Use Case and Class/Object analysis define *what* the system must do, and the Sequence, Activity, State, and Package diagrams define *how* objects collaborate. In an MDD workflow, this PIM would be transformed (manually or via tooling such as StarUML/Enterprise Architect code generation) into Platform-Specific Models (PSMs) — for example, the Component Diagram mapping to specific REST/message-broker technologies and the Deployment Diagram mapping to concrete cloud/on-prem infrastructure — before final code generation. This keeps the domain logic (mission rules, drone state machine) portable and independent of any single implementation platform, and allows the same PIM to target, for example, a cloud deployment or an edge-only deployment for disconnected disaster zones.

12.3 Agile Practices

Given the safety-critical, evolving nature of disaster-response requirements (new drone models, new sensor types, changing rescue-team workflows), ADDRRS is well suited to an Agile, iterative delivery approach: requirements are captured as prioritised use cases (Figure 1) that map naturally onto a product backlog; each sprint can deliver one vertical slice (e.g., "Create and launch a mission" from Figure 2) that is independently testable; the Package/Component structure (Figures 5–6) supports parallel team ownership (one team on Drone Management, another on Alert/Notification) with well-defined interfaces; and frequent stakeholder demos with emergency-response domain experts validate that alert timing, victim-detection accuracy, and dashboard usability meet real operational needs before further investment. Continuous integration and automated regression testing of the drone State Machine and mission workflow are essential given the cost of failure in a life-safety system.

13. Sustainable Development Goals (SDG) Alignment

SDG	Relevance to ADDRRS
SDG 3 – Good Health and Well-being	Faster, autonomous victim location and rescue-team coordination directly reduces disaster casualties and improves emergency medical response times.
SDG 9 – Industry, Innovation and Infrastructure	Demonstrates innovative use of autonomous robotics, edge sensing, and resilient distributed software architecture in critical infrastructure.
SDG 11 – Sustainable Cities and Communities	Strengthens community resilience and disaster-preparedness capacity, a core target of building sustainable, disaster-resilient settlements.
SDG 13 – Climate Action	Supports faster response to climate-related disasters (floods, wildfires, storms), improving adaptive capacity to climate hazards.

14. Reflection

Modelling ADDRRS reinforced how Object-Oriented Analysis and Design provides a disciplined path from an ambiguous real-world problem statement to a precise, implementable design. Identifying actors and use cases first (CO3) grounded every later diagram in genuine stakeholder needs rather than speculative technical structure. Developing the State Diagram for the Drone was particularly instructive: it forced explicit consideration of failure modes (low battery, communication loss) that are easy to overlook in a purely happy-path Sequence Diagram, and it is exactly this kind of behavioural completeness that safety-critical systems demand. Applying GRASP principles and patterns such as State, Observer, and Mediator (CO2) showed how abstract design guidance translates into concrete decisions — for instance, choosing the State pattern for the Drone was a direct, traceable consequence of the diagram in Figure 4 rather than an arbitrary choice. Finally, tracing every diagram back to the same set of use cases and classes (Section 10) highlighted why consistency across UML views is not a formality but the mechanism by which the design's correctness can actually be checked.

15. Conclusion

This assignment analysed and modelled the Autonomous Drone-Based Disaster Response and Rescue System (ADDRRS) using a complete Object-Oriented Analysis and Design approach. Actors and use cases were identified and used to derive a consistent set of domain classes, attributes, methods, and relationships (CO3). Eight UML diagrams — Use Case, Sequence, Activity, State, Package, Component, and Deployment, together with the structural class/object analysis — were produced to capture the system's structure, behaviour, and deployment architecture from complementary viewpoints, with explicit traceability maintained back to the originating use cases (CO2). Design axioms, GRASP principles, and appropriate design patterns (State, Observer, Strategy, Mediator, Facade, Publish-Subscribe) were applied to achieve low coupling, high cohesion, flexibility, and maintainability, while security, reliability, and scalability were addressed as first-class design concerns rather than afterthoughts. Finally, the design was situated within the broader Object-Oriented Systems Development life cycle, Model-Driven Development, and Agile delivery practices (CO1), showing how the same analysis and design artefacts support both disciplined engineering and iterative, real-world delivery of a life-critical disaster-response system.